



機電整合與虛擬實境實驗

Mechatronics and XR Lab 2026

Lab 1

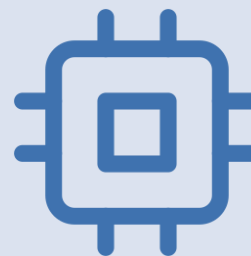
讓 LED、按鍵、可變電阻動起來

Assistant Professor

洪 境晨 Jing-Chen Hong

Teaching Assistant

黃 孟萱 Meng-Hsuan Huang



Today's Five Tasks

From a single LED all the way to a combined application



1 Basic Task 1-1 Blinking LED

① Blinking the built-in LED ② Blinking the LED on the breadboard



2 Basic Task 1-2 Digital Input

Lighting up the LED when a switch is pressed



3 Basic Task 1-3 Analog Output (PWM)

Adjusting the brightness of LEDs by PWM.

Bonus Task: 使用示波器量測並解釋PWM訊號原理



4 Advanced Task 1-1 Analog Input

Checking the value of the variable resistor connected to the analog input from a PC using a serial monitor.



5 Advanced Task 1-2 RGB LED Lighting

Changing the color of RGB LED using various inputs

Three Things to Remember Before You Start



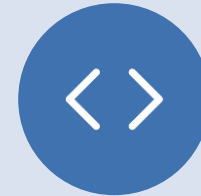
Don't forget the resistor

Every LED needs a current-limiting resistor, or it can burn out instantly



Breadboard top/bottom halves

The middle gap may not be connected — check before wiring



Task check & submission

Every task must be checked by a TA — see the next slide for details

限流電阻!



Task Check & Submission Rules

You must complete the task check, and upload your files for Advanced Tasks



For every task

- **All tasks** must be checked by the TAs — not just Advanced Tasks
- Call the TA over and briefly explain your sketch and circuit yourself
- The TA may ask follow-up questions about how your code works
- **AI may not be used to generate code** — but you can use it to look up commands, examples, or circuits (verify anything you find against official sources)



For **Advanced Task** submission

- Upload the **sketch file** and a **demo video** to GitHub — after your task check
- Create a "Lab 1" folder. Suggested naming: "Task1-1.ino", "Task1-1.mp4"
- Confirm with the lecturer or TA that your files uploaded successfully before you leave the classroom
- **AI tools** are allowed to help write your sketch — but you must understand it and be ready to explain it to the TA

Commands and Variables

To finish the tasks today, you'll need to use the following commands and variables.

void pinMode(pin, mode) :

Set terminal operation mode

void digitalWrite(pin, value) :

Set the output of the terminal

int digitalRead(pin)

Read the value input from the terminal (returned as “int” value)

- pin: pin number (2 to 13, A0 to A5)
- mode: 0 = INPUT, 1 = OUTPUT
- value (output mode): LOW (0) / HIGH (1)
- value (input mode): built-in pull-up ON (1) / OFF (0)

void analogWrite(pin, value) :

PWM signal output

- pin: pin number (3,5,6,9,10,11)— **Why?**
- value: PWM duty ratio (0 ~ 255)

int analogRead(pin) :

Reading the input voltage (returned as “int” value)

- pin: pin number (0 ~ 5 : a0 ~ a5 pins)
- return value: Analog value 0 ~ 1023 — **Why?**

Basic Task 1-1 Blinking the LED

🎯 Demo goal

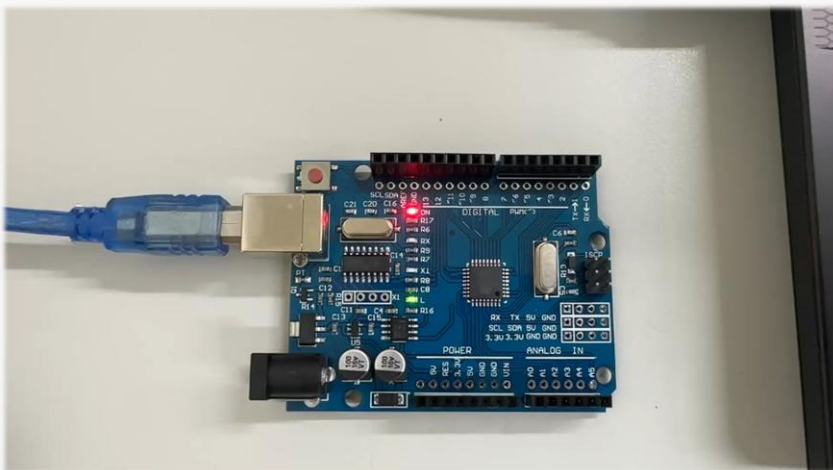
- ① Blinking the built-in LED
- ② Build your own circuit and blink an LED on the breadboard. Blink period = 1 second.



Hint

- ① Check the output pin number of the built-in LED.
- ② Find a breadboard LED-blink example online —
remember the current-limiting resistor! 限流电阻!

Change the pin number definition to switch between the execution of ① and task ②.



```
const int led_pin = ____;
```

```
void setup() {  
    pinMode(led_pin, ____);  
}
```

```
void loop() {  
    digitalWrite(led_pin, ____);  
    delay(____);  
    digitalWrite(led_pin, ____);  
    delay(____);  
}
```

? Why?

Why must pinMode be called before digitalWrite?

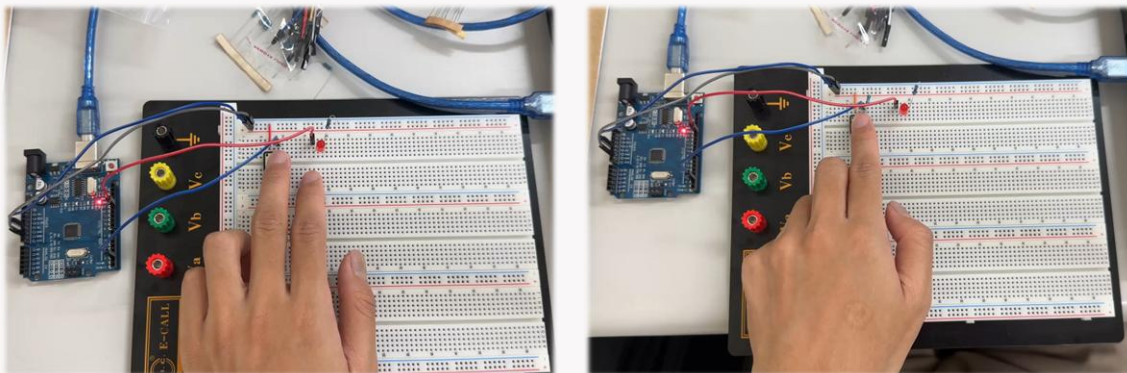
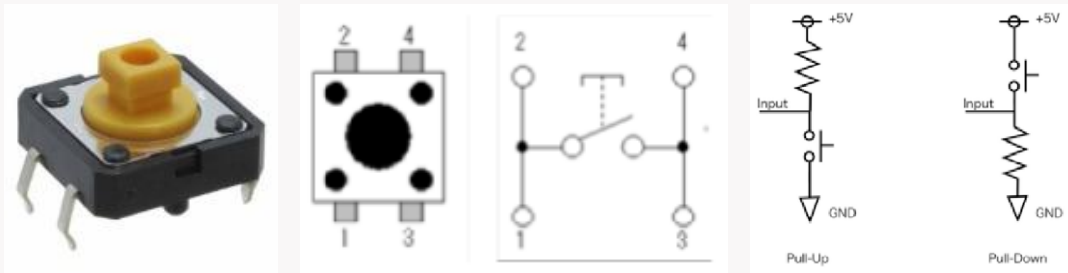
Basic Task 1-2 Button-Controlled LED

🎯 Demo goal

Lighting up the LED when a push button switch is pressed

💡 Hint

You can use either pull-up or pull-down resistor to design and build your circuit for switch connection.



```
const int buttonPin = 2;
const int ledPin = 13;
int buttonState = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, ____);
}

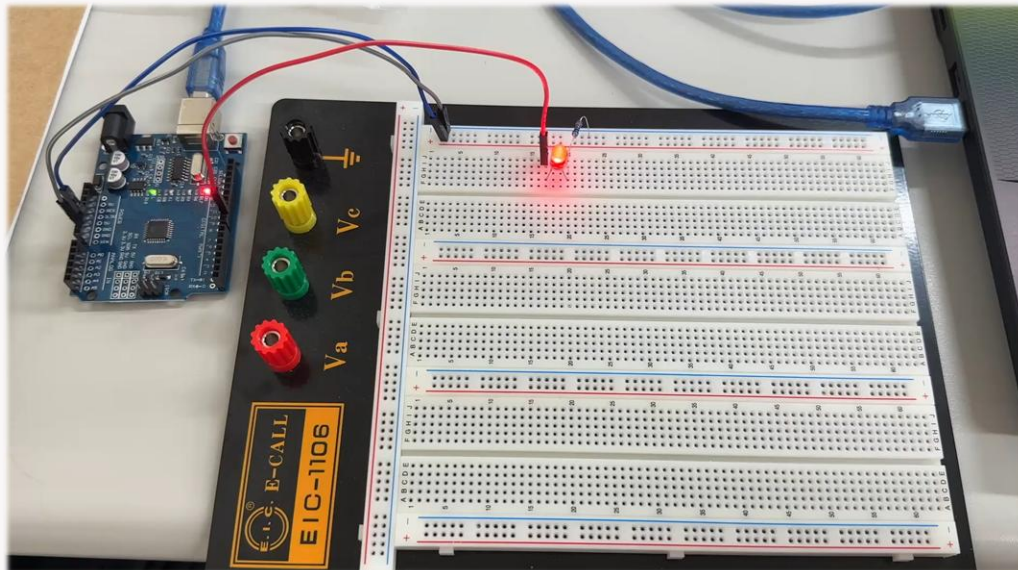
void loop() {
  buttonState = digitalRead(buttonPin);
  if (buttonState == ____) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

Basic Task 1-3 Adjust brightness of the LED by PWM

🎯 Demo goal

Use `analogWrite(pin, value)`, and be ready to explain how it adjusts brightness during your task check.

Bonus Task: Use the oscilloscope to explain the relation between PWM and LED lightness during task check.



```
const int ledPin = ____;

void setup() {
  pinMode(ledPin, ____);
  analogWrite (ledPin, ____);
}

void loop() {
}
```

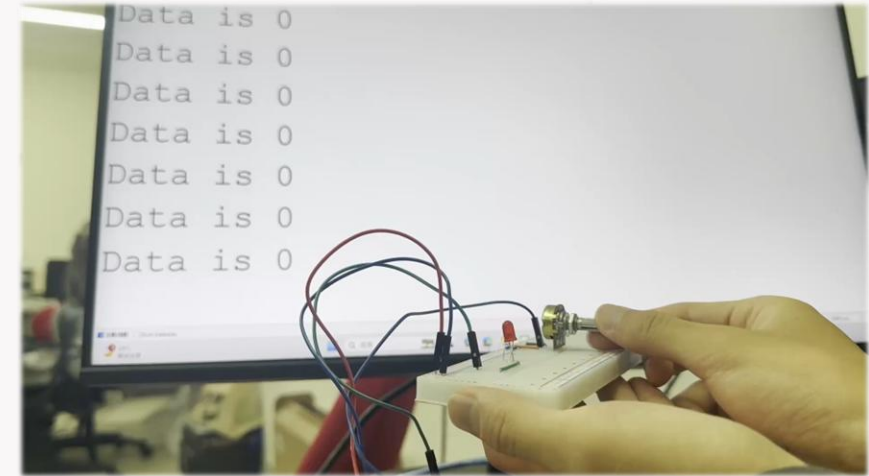
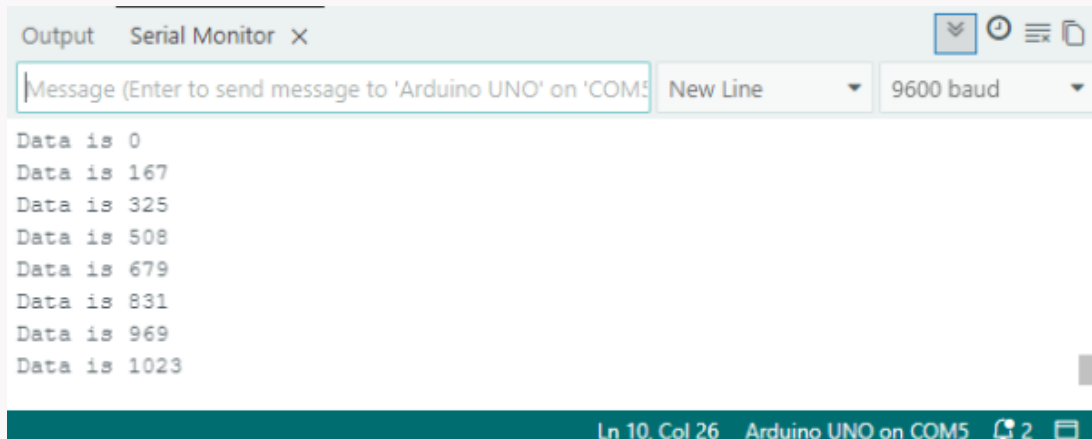
❓ **Why?** When you demonstrate this task, please also explain:

Which analog pins can you use for finishing this task? Why?

Advanced Task 1-1 Using the variable resistor to adjust LED lightness

🎯 Demo goal

- You have to use a variable resistor, and use the “int analogRead(pin)” to read the return value
- Adjust the LED's brightness based on the analog value from Task 1-3 — the larger the value, the brighter the LED.
- Meanwhile, print the analog return value every 1 second in the Serial Monitor as follows:



Hint

The commands for serial monitor use:

`Serial.begin(baud);`

`Serial.print(val);` or `Serial.println(val);`



Why? When you demonstrate this task, please also explain:

What is the maximum of “int analogRead(pin)” return value? Why is it?

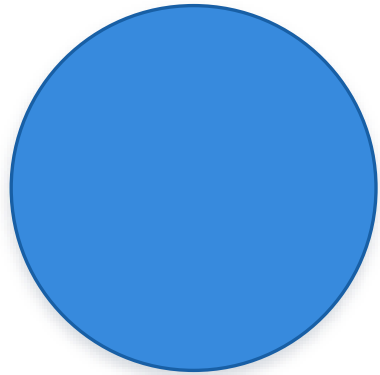
Advanced Task 1-2 RGB LED — Layering Three Inputs

Control an RGB LED with three inputs, each layering on top of the last

● R = Button (digital input)

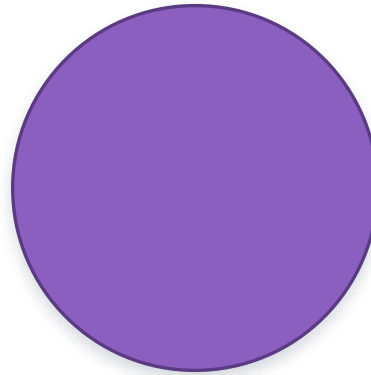
● G = Serial input (from the Serial Monitor)

● B = Variable resistor (analog input)



Variable resistor is dialed

Blue



+ press button

Purple



+ serial input "1"

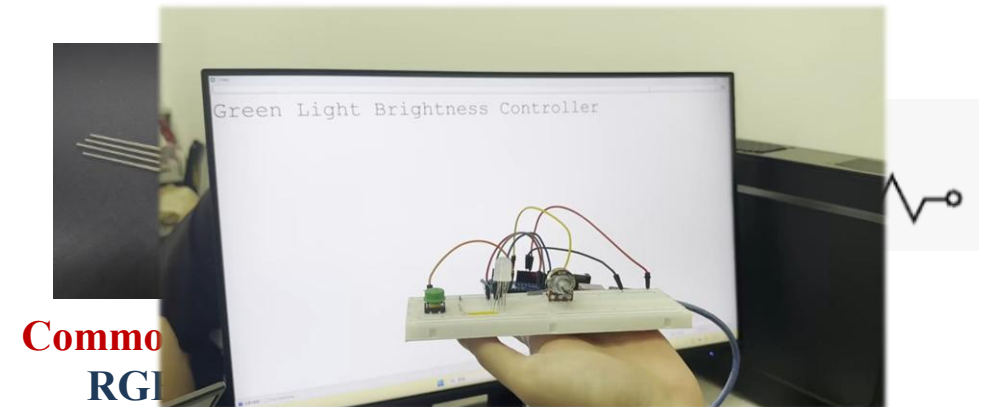
White

❓ Hints

所有color pin都各需要接一個限流電阻!

- All of these are what we have done in basic tasks. Just combine them!
- When you get one color turned on, move on to the next.
- Connect the longest pin of the LED to the GND pin of the Arduino board.

The current limiting resistors are required for all color pins of the LED.



Stuck? Troubleshooting Checklist



LED won't light up

Check polarity (long leg = positive) and the resistor



Button logic reversed

INPUT_PULLUP reads LOW when pressed; a pull-down reads the opposite



Wrong PWM pin

Only pins 3,5,6,9,10,11 support analogWrite



RGB colors look wrong

Check whether it's common-cathode or common-anode — wiring is reversed

Lab 1 Complete!

Next → Lab 2: Motors, Servos & Sensors

Q & A